SR-MPLS Training Lab

M4 Traffic Engineering (Flexible Algorithm)

Estimated time: 75 min

Objective

Build two independent shortest-path planes out of one IS-IS topology, each computed by a different Flexible Algorithm, then steer a customer VPN onto the one you want. Flex-Algo 128 optimizes on the IGP metric and follows plane A; Flex-Algo 129 optimizes on measured link delay. You will define both algorithms, bind them to colored transport-classes so they resolve in inetcolor.0, tag CUST_B with the color-129 community so its VRF rides the delay plane, then use GNS3 to inject about 80 ms of latency on a plane-A link and watch algo-129 recompute and move CUST_B onto plane B while CUST_A on algo-128 does not move.

Before you start: M3 complete: the iBGP overlay is up through the C1 and C2 route reflectors and both L3VPN services (CUST_A target:64512:100, CUST_B target:64512:200) are established with CE reachability on every PE.

Concept primer

A Flexible Algorithm is a separate SPF calculation that runs alongside the default algo-0 shortest-path tree. Each definition names a metric type to minimize and, optionally, link constraints to include or exclude, and every router in the domain that participates advertises which algorithms it supports plus a distinct prefix-SID per algorithm for its loopback. Because each algorithm has its own SID and its own SPF result, one physical topology yields several logically distinct forwarding planes at once. In this lab algo-128 is defined with metric-type igp-metric, so it reproduces the familiar lowest-IGP-cost path and follows plane A; algo-129 is defined with metric-type delay-metric, so it minimizes cumulative measured one-way link delay instead of IGP cost.

The two planes only differ when the metrics differ. Every core link is IGP metric 10, so algo-128 picks plane A (PE1-P1-C1-P3-PE2) as the lowest-cost path to PE2. Algo-129 instead sums the delay values that IS-IS distributes as APP-LRTT sub-TLVs, which come from the delay-measurement probes each transit interface runs. When every link reports a low, similar delay the two algorithms agree, but the moment one plane-A link starts reporting a large delay, algo-129's SPF re-

solves and prefers the lower-delay plane B while algo-128 stays put because IGP metrics have not changed. That is exactly the cause and effect you will trigger by hand later in this module.

To make traffic actually use an algorithm you connect three pieces. First, 'use-transport-class' on each flex-algo definition, plus a transport-class named for its color (color 128, color 129), causes each algorithm's SR tunnels to be installed in the colored transport RIB inetcolor.0 keyed by color rather than by plain loopback. Second, a BGP VPN route that carries an extended color community (color:0:129) is resolved through the COLOR-RESOLUTION-MAP in ip-color mode, so instead of resolving over the default inet.3 tunnel it resolves over the matching colored entry in inetcolor.0. Third, on CUST_B the import policy SET-COLOR-129 stamps every CE route with color:0:129, so the whole VRF rides algo-129 while CUST_A, which carries no color, keeps resolving over algo-128. Steering is therefore per-VRF and driven entirely by a community, with no per-flow configuration.

Build steps

1. **Define both Flexible Algorithms and their colored transport-classes.**

```
set routing-options flex-algorithm 128 definition metric-type igp-metric
set routing-options flex-algorithm 128 color 128
set routing-options flex-algorithm 128 use-transport-class
set routing-options flex-algorithm 129 definition metric-type delay-metric
set routing-options flex-algorithm 129 color 129
set routing-options flex-algorithm 129 use-transport-class
set routing-options transport-class auto-create
set routing-options transport-class name ipv4-tc-128-igp color 128
set routing-options transport-class name ipv4-tc-129-delay color 129
```

Note: algo-128 minimizes the IGP metric, algo-129 minimizes measured delay. 'use-transport-class' plus a color-matched transport-class is what lands each algorithm's tunnels in inetcolor.0 keyed by color 128 or 129. These same nine lines are identical on every SR node.

2. **Advertise participation in both algorithms in IS-IS.**

```
set protocols isis source-packet-routing flex-algorithm 128
set protocols isis source-packet-routing flex-algorithm 129
```

Note: This tells the domain that this router computes algo-128 and algo-129 and will honor their prefix-SIDs. A node that does not advertise an algorithm is pruned from that algorithm's SPF tree.

3. **Advertise a distinct prefix-SID per algorithm for this loopback.**

```
set policy-options policy-statement EXPORT-FLEX-ALGO-SID term sid from interface lo0.0
set policy-options policy-statement EXPORT-FLEX-ALGO-SID term sid from route-filter 10.0.0.1/32 exact
set policy-options policy-statement EXPORT-FLEX-ALGO-SID term sid then prefix-segment algorithm 128 index 1001
set policy-options policy-statement EXPORT-FLEX-ALGO-SID term sid then prefix-segment algorithm 128 node-segment
set policy-options policy-statement EXPORT-FLEX-ALGO-SID term sid then prefix-segment algorithm 129 index 2001
set policy-options policy-statement EXPORT-FLEX-ALGO-SID term sid then prefix-segment algorithm 129 node-segment
set policy-options policy-statement EXPORT-FLEX-ALGO-SID term sid then accept
set protocols isis export EXPORT-FLEX-ALGO-SID
```

Note: Each algorithm needs its own SID index so the two planes never share a label. The convention in this lab is algo-128 index 1000+node and algo-129 index 2000+node: PE1 (node 1) uses 1001 and 2001, P1 (node 21) uses 1021 and 2021. The absolute label is SRGB-start 16000 plus the index. Change only the two index values per node.

4. **Turn on delay measurement so algo-129 has a metric to minimize.**

```
set protocols isis interface ge-0/0/1.0 delay-measurement probe-interval 1
set protocols isis interface ge-0/0/1.0 delay-measurement probe-count 3
set protocols isis interface ge-0/0/1.0 delay-measurement advertisement periodic threshold 90
set protocols isis interface ge-0/0/1.0 delay-measurement advertisement periodic interval 60
set protocols isis interface ge-0/0/2.0 delay-measurement probe-interval 1
set protocols isis interface ge-0/0/2.0 delay-measurement probe-count 3
set protocols isis interface ge-0/0/2.0 delay-measurement advertisement periodic threshold 90
set protocols isis interface ge-0/0/2.0 delay-measurement advertisement periodic interval 60
```

Note: Each transit interface probes its link, then advertises the measured one-way delay as an IS-IS APP-LRTT sub-TLV that algo-129 sums. Without an active delay-measurement stanza on a link, algo-129 has no delay value for it and treats that link as infinite delay. Apply on every transit interface on the node: P1 has three (ge-0/0/1, ge-0/0/2, ge-0/0/3), PE1 has two.

5. **PE only: enable color resolution and steer CUST_B onto algo-129.**

```
set policy-options resolution-map COLOR-RESOLUTION-MAP mode ip-color
set policy-options policy-statement VPN-COLOR-RESOLUTION term map then resolution-map COLOR-RESOLUTION-MAP
set protocols bgp group OVERLAY import VPN-COLOR-RESOLUTION
set policy-options community COLOR-129 members color:0:129
```

```
set policy-options policy-statement SET-COLOR-129 term t1 then community add COLOR-129
set policy-options policy-statement SET-COLOR-129 term t1 then accept
set routing-instances CUST_B protocols bgp group CE import SET-COLOR-129
commit and-quit
```

Note: This block lives only on the PEs (PE1, PE2), not on P or C routers. The ip-color resolution-map on the OVERLAY session lets colored VPN routes resolve over inetcolor.0. SET-COLOR-129 tags every CUST_B CE route with color:0:129, so the whole CUST_B VRF rides algo-129. CUST_A carries no color and keeps resolving over algo-128.

6. **In GNS3, inject latency on a plane-A link (no router config).**

```
## GNS3 GUI action, not a Junos command:
## right-click the PE1 <-> P1 (plane A) link > Packet filters...
## add a 'delay' filter of ~80 ms, apply, and OK
## commit nothing on the routers: this is a GNS3 link filter
```

Note: A GNS3 per-link delay filter adds latency to the frames the IS-IS delay-measurement probes ride on, so P1 and PE1 begin advertising a much larger APP-LRTT value for that link. Algo-129 re-solves and prefers plane B; algo-128 is unchanged because the IGP metric is still 10. This is the learner-triggered cause and effect at the heart of the module. Remove the filter afterwards to restore the baseline.

Verification

show route table inetcolor.0

```
10.0.0.2-128<c>/64
                *[L-ISIS/14] 00:15:32, metric 40
                 >  to 10.1.1.2 via ge-0/0/1.0, Push 16002
10.0.0.2-129<c>/64
                *[L-ISIS/14] 00:15:32, metric 300
                 >  to 10.1.1.2 via ge-0/0/1.0, Push 16002
```

What to look for: Two colored transport routes to PE2 exist: the -128 entry (IGP metric 40) and the -129 entry (summed delay). Before latency injection both leave via the plane-A next-hop 10.1.1.2 (ge-0/0/1 toward P1). A missing -129 entry means algo-129 has no delay data on some link and has not converged.

show isis database extensive | match delay

```
IS extended reachability delay: 100 usec (ge-0/0/1.0)
IS extended reachability delay: 100 usec (ge-0/0/2.0)
 ... one advertised delay per transit link, all planes present
```

What to look for: Every transit link that runs delay-measurement advertises an APP-LRTT delay sub-TLV. All plane-A and plane-B links should show a value. A link with no delay line is invisible to algo-129 and is treated as infinite delay, which is exactly this module's break/fix.

show route table CUST_B.inet.0 detail

```
192.168.20.0/24 (1 entry, 1 announced)
     *BGP    Preference: 170/-101
          Communities: target:64512:200 color:0:129
          Next hop type: Indirect, resolved over inetcolor.0
          Push 16002  (via the algo-129, color 129 tunnel)
```

What to look for: CUST_B's remote CE prefix carries color:0:129 and resolves over the inetcolor.0 algo-129 tunnel, not the default inet.3 tunnel. This proves the VRF is steered onto the delay plane. If Communities shows no color, SET-COLOR-129 did not run and CUST_B has fallen back to algo-128.

show route table inetcolor.0 10.0.0.2-129<c>/64 (before, then after GNS3 latency)

```
before:  *[L-ISIS/14] metric 300
          >  to 10.1.1.2 via ge-0/0/1.0, Push 16002   (plane A, toward P1)
after:   *[L-ISIS/14] metric 80300
          >  to 10.1.1.18 via ge-0/0/2.0, Push 16002  (plane B, toward P2)
```

What to look for: Once the ~80 ms GNS3 filter is applied to the PE1-P1 link, algo-129's summed delay to PE2 jumps and the -129 next-hop flips from ge-0/0/1 (plane A, P1) to ge-0/0/2 (plane B, P2). Re-run 'show route table CUST_B.inet.0' and CUST_B now follows plane B, while the -128 entry and CUST_A stay on plane A. Remove the filter to watch it flip back.

[screenshot 005: M4 verification: CUST_B follows the delay-optimized algo-129 path]

Break / Fix exercise

Scenario: CUST_A is healthy and its route in inetcolor.0 resolves normally over the algo-128 (color 128) tunnel to PE2. CUST_B, which should ride the delay-optimized algo-129 plane, is misbehaving: on PE1 the 10.0.0.2-129 transport route in inetcolor.0 is either missing or has taken a strange detour, and 'show route table CUST_B.inet.0 detail' shows CUST_B no longer following the expected delay path toward PE2. Every IS-IS adjacency is Up, every BGP session is Established, and algo-128 is completely unaffected, so nothing at the adjacency or session layer looks wrong.

Your task: Explain why only algo-129 (and therefore only CUST_B) is broken while algo-128 and CUST_A are fine, find the single transit link that is not contributing to the delay plane, and restore normal algo-129 routing so CUST_B returns to its correct delay-optimized path. Fix only what is broken and do not touch the immutable mandates or the IGP metrics.

Hint: Algo-128 minimizes the IGP metric and algo-129 minimizes measured delay, so a fault that only hurts algo-129 has to be in the delay data, not the IGP topology. Compare the advertised delay sub-TLVs across all transit links: 'show isis database extensive | match delay' will show every link reporting a delay except one. Walk the plane-A links, find the interface whose delay measurement is not being advertised, and get it advertising again. A link with no delay value is treated as infinite delay by algo-129.

The instructor holds the fault-injection and fix commands in the answer key. Diagnose it yourself first.

Brief it back

Explain each of these aloud before moving on. If you cannot, re-read the primer and re-run the verification commands.

- Explain how a Flexible Algorithm builds a separate SPF plane from the same physical topology, and trace the full path by which a single BGP color community steers the CUST_B VRF onto algo-129 instead of algo-128.
- The delay advertisement for one plane-A link went missing, breaking algo-129 and CUST_B, yet algo-128 and CUST_A kept working perfectly. Explain precisely why the two algorithms responded differently to the same failed link.
- Describe how adding an ~80 ms GNS3 delay filter to a single link changes the IS-IS delay metric but not the IGP metric, and why that is enough to move CUST_B from plane A to plane B while leaving CUST_A where it was.